Summarize

Chapter 16. Common Failure Patterns

So far this book has covered various patterns to help you build your distributed system. This chapter is going to be a little different. Instead of helping you know what to do, it is intended to help you know what *not* to do. Over numerous years of developing, operating, and debugging systems, certain kinds of problems repeat themselves. These patterns are divided into mistakes that are made in building the systems, as well as common ways in which systems fail. By understanding both what not to do and what to try to prevent, we can learn from these shared mistakes and prevent them from repeating in the future.

The Thundering Herd

The thundering herd derives its name from the metaphor of a bison or other large animal on the prairie. Individually they may be manageable, but when moving together, charging, they are capable of destroying anything they are directed toward. The easiest way to understand the thundering herd is to imagine yourself interacting with a website that is not behaving properly. You attempt to navigate to a particular location, the loading progress bar spins slowly, not making very much progress, eventually you become impatient and you hit the reload button. You may not know it, but you have become the thundering herd.

Any particular application has a maximum capacity. Typically we try to size our applications so that its maximum capacity is greater than any load that it experiences, even at its most busy. Unfortunately, sometimes, because of failures, lost capacity, or unanticipated demand, the requests sent to a service exceed its ability to service those requests. Let's call the size of this overload traffic X requests per second. Being overloaded is bad; many, if not all, of those X requests per second time out or fail, but what happens next is even worse. In the next minute, those same X requests become 2X requests. This doubling is the result of any net new traffic (X) as well as all of those previous requests being retried. The result is even more failures, even more retries, and even more new requests. What was already a bad situation has become irrecoverable. This is especially

true when there are multiple chained calls and multiple retry algorithms involved. The amplification of one request into many requests can rapidly overwhelm a system.

When we build our systems, we build in retry in order to handle when requests fail, but if we blindly retry when we see errors, we make the thundering herd worse and worse. On the client side, the first way to handle this is to use exponential backoff. Instead of immediately retrying, the client waits a period of time before trying again. Every time the client sees an error, it doubles the time it waits.

Exponential backoff is good, but it is imperfect; in addition to exponential backoff, it is also very useful to add *jitter*. Jitter is a small amount of randomness which spreads load across time. Without jitter, a situation can occur where everyone fails at the same time, waits the same amount of time, and then hammers the service again at the same time. Jitter is especially important in situations where programs are talking to other programs. Humans are sufficiently random that they add their own jitter, but unfortunately, machines are not.

The final and most comprehensive way to solve the thundering herd is to add a *circuit breaker* to your client. The circuit breaker is something that trips when error rates exceed some threshold and stops all traffic for a period of time, giving the system time to recover. Finally, to aid in recovery, sometimes it is also necessary to only gradually let traffic back into the system. Many systems can be stable under high load but must be brought up to that load level gradually. Dumping 100% of the traffic on a server all at once can simply reintroduce the thundering herd.

The Absence of Errors Is an Error

We will move from problems of too much traffic to problems caused by too little traffic. When building a system, it is common to build monitoring and alerts to fire when there are too many errors in the system. Obviously, too many errors is a problem that requires a person to look into the system, determine the cause of the errors, and restore the service. But what about too few errors? Though it can seem counter-intuitive, some small number of errors is the steady state for most distributed systems. It is generally very hard and not worth the effort to prevent any errors from ever occurring; thus, there is a low level of ambient errors that are handled by retry and other error-correction systems. Consequently, the absence of *any* errors is more likely to indicate a serious problem rather than that everything is awesome.

To understand how this can happen, imagine monitoring a system which adds subtitles to movies that are uploaded to a storage bucket. Whenever a movie is uploaded, the system performs speech to text and translation to add subtitles and then stores the new movie back into the storage bucket. Imagine what happens when the process that uploads movies accidentally loses permissions to upload those movies. If you are only monitoring for conditions where errors are greater than some percentage (say, 10%), then you will never alert for this condition. If the uploader cannot upload movies, then no movies are being processed, so no errors can occur, so your error rate is 0%. But clearly, if your uploader can't upload movies, your system is broken. Alerting on both too many and too few errors (or too few requests generally) will enable you to catch (and fix) such problems in your systems.

"Client" and "Expected" Errors

The next pattern considered is also related to errors in the system, but this time related to ways in which we feel OK ignoring errors. Many systems take in user input and requests for processing. In such systems, it is possible for users (or clients generally) to send requests which are invalid or unprocessable in some way. When calculating the reliability of a service, it's clearly problematic to treat errors caused by bad client requests as true errors, since the service isn't in control of what the clients send. While this is true in general, in practice such "not my problem" buckets become convenient places to ignore true system errors. Imagine, for example, something as simple as an authorization error. Certainly a user not being authorized to make a request is a "client" error and doesn't indicate a problem with reliability. But what if suddenly *all* users are unauthorized? That's probably a problem with our authorization code. But if you treat all unauthorized errors as "user errors" and you don't look for anomalies like all users failing authorization, you probably won't alert when you break your authorization code.

The same thing can be true of "expected" errors. I mentioned in "The Absence of Errors Is an Error" that there's a certain level of errors that are expected in the system, but for synthetic requests that you create, that's not true. When monitoring a system, it is also important to monitor real client requests where some small percentage of errors is expected and also to write *synthetic* or system-generated requests, which you control entirely. With synthetic requests, you control both the client and server side of the interactions and, consequently, there should never be any "client" or "expected" errors for those requests. By combining monitoring of real traffic with monitoring of synthetic requests, you can quickly identify situations where your code changes are causing errors which would otherwise be ignored as client errors.

Versioning Errors

Whenever you are building software, you are going to need to iterate and make changes to that software; but clients require and expect consistency in how they interact with your systems. If you need to update every client at the exact same time as you update your system, you haven't actually built a decoupled, distributed system. Any system is made up of the external API that it exposes to clients and the internal in-memory representation of that API. At first glance, it seems that the best solution is to make these two representations the same. If they are different, then translation logic needs to be applied to take the external representation to the internal representation and vice versa. And of course, when you start out, the external and internal representations are identical.

In practice, it turns out that separating the internal on-disk or in-memory representation of your API from the external client-perceived representation of your API pays immense dividends. For starters, it makes it relatively easy to have multiple different external client API versions implemented by the same internal in-memory version. This means that different clients can talk to you using different versions of your APIs, and you don't need to tightly synchronize your clients and your application. Additionally, the separate internal representation means that it is easier for you to iterate on your API implementation; adding, removing, or merging API fields as necessary to support your application can be done without the client noticing as long as you maintain the translation logic between external and internal representations.

Though the benefits of this separation are often seen in supporting multiple clients, the same can be applied to your internal storage versioning. Ultimately, your application will also need to store the objects in your API in some data store. This storage also needs to be versioned. If it is tightly coupled with a specific version of your code, it can make it very difficult to roll out changes to your storage layer, or roll back code when there are bugs.

Even though it may seem overly complicated to begin with, building every application with an external (client) version, an internal (in-memory) version, and a storage (on-disk or database) version as well as the logic to translate between them pays significant dividends in decoupling and agility that will benefit your application in the long run.

The Myth of Optional Components

Distributed systems are organic systems. We may design them in one way to start, but over time and in reaction to outages, scale challenges, and business changes, they morph and adapt to meet new requirements. One of the most common changes is the addition of caches in various locations to improve performance and reduce load.

Typically, when these caches are added to the system, they are treated as "optional" components. After all, a cache is just a performance improvement. If the system needs to, it can always obtain the data from the source of truth. The problem with theoretically optional components in your system is that, over time, they become required parts of your design. Consider adding a cache to a system. It is an optional performance improvement when originally added. However, as the load on the system grows over time, its performance becomes more and more dependent on the presence of the cache until the system can no longer meet its performance goals without the cache.

Unfortunately, this increased reliance often goes unnoticed, and the overall reliability of the caches in the system is not maintained in correspondence with its increased criticality to overall system stability.

One of the most common ways that this manifests is in the difference between a local and a global cache. A local cache, as the name suggests, is a cache that is maintained locally in the memory of your application, often as a simple hash table. Such caches are very easy to implement. Unfortunately, as the system grows, the overall size of such local caches has to grow proportionally, and eventually the overall memory demands of the system become too great to sustain the cache. Furthermore, there is a tendency to add many such in-memory caches within a single application because it is so easy to do. Consequently, the memory usage of such caches is difficult both to visualize and manage.

Additionally, as discussed in Chapter 15 on caching, local caches are very bad at optimizing memory usage. Consequently, if you ever find yourself adding a local "optional" cache or any other theoretically "optional" component to your system, accept that you are actually making a permanent (and very required) change to your overall distributed system that deserves the attention of any other significant change or refactor.

Oops, We "Cleaned Up" Everything

In any real system, it's an unfortunate truth that errors occur. Bugs and failures are bound to happen. Given this, it is often important to consider how errors and failures might impact the overall state of the system that we are building. When doing this, it is important to think about more than just the operation of the individual pieces that make up the system, but also to think in terms of how all of these pieces interact together to manage the entire distributed system. Failing to anticipate how these systems might interact together in the presence of a failure is a common cause of severe problems. One of the most severe is a runaway deletion of a system because of some sort of unanticipated interaction effect.

Deleting everything in your system seems like such an extreme failure that it is hard to understand how it might happen by accident via a minor bug. Consequently, it is useful to examine real-world examples of how such a thing might happen.

An example is a photo storage system. Imagine that it has four components:

- A database that stores user information
- A frontend RESTful API server that implements the user API
- A file storage system where all of the photos are stored
- A garbage collection system, which cleans up photos for users after their accounts are deleted

In normal operation, a user initiates an account deletion through the website, and the user is removed from the database. Then, for each photo stored in the filesystem, the garbage collection system looks up the user in the database. If the user can't be found, the photo is deleted. This garbage collection system exists to keep the system compliant with data privacy laws as well as to ensure photos don't "leak" and cause infinite growth in the filesystem.

Now consider a piece of code written in the frontend RESTful API to look up a user. It initiates a connection to the database, looks up the user's information, and then returns it to the caller. But what if the database is unavailable for some reason? Suppose the person who coded that piece of logic chooses to return a 404 indicating that the user couldn't be found. "Not Found" is perhaps not the best error to return in this case—500 internal service error is definitely better; but it is hard to see how this error code is important until you think about the garbage collection system. When the garbage collection system inquires about *any* user, they will all appear to be missing, and the garbage collection system will delete *all* images. Fortunately, because people anticipated cata-

strophic problems, the photos were restored from backups; but the service was down for many hours while these backups were restored from cold storage. During this period, user confidence in something so deeply personal and important as your family photos was shaken, and the service had to work very hard to restore confidence.

So what went wrong? There are secondary basic failure points in this system that caused the catastrophic failure. The first is a human failing. Namely, it is hard for any human to keep the full context of a distributed system in their head at all times. Especially in a situation where one team may have owned the user API system, while a different team, possibly in a different geographic location, owned the garbage collection system. It is tempting to focus on this human failure. The developer made a mistake and "caused" this outage. Unfortunately, human failures are an inherent part of distributed systems; and ultimately they cannot be fixed, only prevented. Humans, regardless of their seniority, will make mistakes. Our systems must anticipate and prevent the effects of those mistakes.

This leads us to the second failing: the process or technical failure in the system, namely, rate limiting and a circuit breaker in the garbage collection logic. In the system under normal operation, there is always roughly the same number of users deleting their accounts. Let's say that one percent of all users delete their accounts in a given day; this will lead to a relatively constant rate of image deletes. When this bug occurred, the rate of image deletes increased by 100 times. This is clearly unusual, and the system should have detected and prevented such a high rate of deletes. The first systematic improvement would be the addition of throttling to the deletes. The throttling sets an upper bound on deletes per second and would have dramatically slowed down the rate at which all of the photos could be deleted to give humans a better chance to react and stop the process. The second is a "circuit breaker," much like the circuit breakers in our houses; when this high rate of deletes was hit for an extended period of time, the garbage collection system should have stopped itself from performing *any* deletes until a human reset the breaker. Combined, while these technical changes couldn't prevent there being some impact from the human mistake, they would dramatically reduce the impact on both the customers of the service and the brand of the service itself.

Such accidentally "we cleaned up everything" scenarios are actually surprisingly common, because as developers we tend to focus on the happy path where everything is working correctly. In that world, the call to the user API can never return a wrong answer. Experience in distributed systems teaches us that we always need to be thinking about a world where things are going right, but also what happens when things go wrong.

Challenges with the Breadth of Inputs

One of the most disappointing kinds of failures is the failure that slips through all of your testing and goes undetected until a customer notices it. Such failures are frustrating to your customers. "Don't you test your code?" is a hard question to answer well in such cases. They are equally embarrassing because often they sit in your codebase for a long time before someone notices and/or complains loud enough to get you to notice.

Sometimes these errors are undetected because it takes a very rare confluence of events to make them happen, e.g., tricky race conditions that occur once in a blue moon. Such bugs are actually significantly less likely to frustrate an end user; they just look like flakiness. We have all been trained to "reload and try again," which generally fixes such bugs.

The bad ones are the ones which occur for specific users 100% of the time, but only for a small number of users. I typically call these problems "compiler" problems, because they typically occur when there is a very broad set of allowed inputs to our system. In such a system, very specific combinations of input can cause problems 100% of the time, but only certain users present such inputs.

These are called "compiler" problems because the compiler is one of the first programs where such problems manifest themselves. For any given programming language, the space of possible legal programs is effectively infinite. Given enough programmers working on enough programs, sooner or later, one of them will write a program that is by the book legal, but exposes a bug in the compiler that causes it to crash. Sometimes these inputs are the product of random chance and human nature, but more and more often these inputs are created by fuzzing programs, special purpose code that fire random data at APIs in the hopes of causing crashes to deny service or exploit weaknesses in the APIs. An (in)famous version of such attacks were the SQL injection attacks that were typical of the early web, where bits of random SQL (e.g., `'; DROP TABLES'`) were provided as inputs to places where no developer could reasonably expect SQL statements (e.g., a credit card number). Though, by now SQL injection attacks are largely a thing of the past, accidentally breaking legitimate users or randomized "fuzzing" attacks are still quite common.

Ultimately, this failure represents a failure of coverage in testing. If the surface area of a program is completely tested, such failures can't happen; but the space of valid inputs for many APIs is infinite or near infinite. How can you ensure coverage of your testing in such a situation? There are two basic solutions.

The first is to record a representative sample of real-world inputs. This sample can never be exhaustive, but if you record (for example) all user inputs over the last two months, you can be reasonably sure that you are unlikely to break a legitimate user. When you are doing this recording, though, you need to be careful about both where and when you record it. If you have a global service, but only record inputs in North America, you will likely miss many bugs in inputs with Asian characters. Similarly, if your input varies between night and day, and you only record at a certain time, you will miss variation in your inputs due to the time of day. High-quality coverage in testing only comes when your test data is representative of *all* of your users.

The second solution is to use fully randomized or "fuzzed" data for your inputs. These tests can be significantly more comprehensive, but they are also much harder to create. For a simple API that perhaps only takes a few integers as parameters, it is easy enough to generate random numbers, but for more complicated APIs with richer semantics, you may need to write custom random input generation logic (for example, something that generates random but valid IP addresses). Lately, innovation in the space of LLMs and generative AI makes it possible to generate valid but random inputs for even more classes of data.

The combination of testing previously seen inputs as well as using fuzzed data can help ensure that your new code won't break existing users and that it is hardened against possible attacks via random or maliciously crafted command inputs.

Processing Obsolete Work

We've all been faced with an unresponsive computer system: an email system where searching isn't working, a retail site which refuses to load the next page. This chapter is full of the many different ways in which systems can fail to operate correctly. Faced with these failing systems, what do we all do? We hit the *reload* button. And thus, an additional way to fail has been created for our services.

In "The Thundering Herd", we discussed the thundering herd and how retries or failovers can rapidly take down an entire system. The problem discussed in this section has somewhat of the opposite flavor, and that is a system struggling to catch back up.

Imagine a system that is searching for photos that match a photo of our favorite pet. In this system, requests come in, they are processed asynchronously, and the results are made available for the person making the request. But what if, by the time the results come back, the person is no longer waiting? What if they have hit *reload* and resubmitted

one, or even countless additional requests. With no one waiting for the results, the work done to produce the results is wasted, and worse yet, the time taken to produce those results has delayed our system responding to actual user requests.

To understand how this happens, imagine that our photo processing system has suffered a failure in its object recognition microservice. Processing requests grinds to a stop, but the inbound requests keep arriving. A little like a jam in an assembly line, even though no work is being processed, new work keeps coming down the conveyor belt. This is obviously problematic; but working quickly, we manage to resolve the problem, and work starts being processed again. But what happens now?

During the time that the system wasn't processing any work, a backlog of work has accrued at the component which failed. If the system is close to its optimal load at steady state, as is desirable for maximal efficiency, the system will have a very hard time processing through this backlog. As long as the backlog exists, new requests are delayed. It's not uncommon in such situations for a short outage, say less than 10 minutes, to cause a multihour increase in the latency of request processing, and corresponding degradation in customer experience.

So how can we build our systems to respond better in the face of such backlogs and latency increases? There are three approaches which can be used independently or together to solve the situation. The first is timing out requests. Every request that enters the system should have a timeout, the time after which the request is considered invalid and failed. When the request times out, any work associated with that request is aborted. Timeouts can help the system shed work when significantly overloaded. However, it is important to note that timeouts by themselves cannot resolve the problem described above. If the latency of each request is high, but slightly lower than the timeout value, your system will seem very sluggish, but it won't ever shed work to get back to normal.

The second approach is to use autoscaling based on request latency. We are used to seeing autoscaling based on CPU or memory usage, but often it makes even more sense to autoscale based on user-facing metrics like processing latency. If your system can enable autoscaling, scaling based on request latency can help process the backlog that accumulated when it was down. Even simply manually scaling after an outage, while less efficient, can be an effective mitigation.

The final approach is to explicitly prioritize newer requests over older requests when your queues grow too long or latency grows too high. This approach is a form of triage, which sacrifices the experience of some users, who in all likelihood already abandoned

that request anyway, to ensure that more recent requests, likely the same users hitting *reload*, have a better experience.

Designing reliable distributed systems means not only designing systems that prevent failures from occurring, but also systems that recover quickly when failures do occur. Users are far more accepting of short outages when the system recovers quickly than prolonged outages where recovery takes a long time, even after the problem is "fixed."

The "Second System" Problem

So far we have seen numerous common problems that arise in distributed systems. As you have read through this chapter, you may have recognized several different problems that are present in your current system. Seeing the possibility of failures, you are eager to improve the system, but how can you do it?

Sometimes there are only a few changes needed, and it's obvious how to make these minor improvements to the existing system—roll them out and be done with it. Unfortunately, more often than not, the problems seem so large and so hard to fix in the existing system that it seems the most efficient solution is to build a new system, from scratch, that replaces the existing system at some date in the future. This approach is attractive for a number of reasons: it's always easier to build from scratch rather than refactor; it is always more exciting to build something new than it is to refactor an existing solution; and finally, building the new system alongside the old system allows the new system to be built without the pressures of being used in production until it is ready.

The attraction of all of these perceived benefits makes building the "second system" one of the most common failures in distributed systems; but it is this last phrase, "until it is ready," that turns out to be the curse of the second system. In practice, many second systems never actually become ready. There are many different organizational reasons why this can happen, but the most common are that the second system is forever playing catch-up to the current system and that it never gets the production mileage necessary to replace the original system.

The second system tends to play catch-up precisely because it is not the current system in production. This means that while it is being built, all of the important features, bug fixes, and improvements necessary to run the business land in the original system. Suppose it takes six months to complete the second system. At the moment when it is "complete," there are six more months of features and fixes that it needs to implement

until it matches the capabilities of the original system. Unless you are willing to significantly over-staff the team building the second system, you are always playing catch-up.

The need for over-staffing the development of the second system leads to the other failing, which is the production readiness of the second system. In order for any system to become production ready, it needs mileage on it. No matter how good you think your code is, I guarantee that there are bugs that will only be found once it is released into production. But when bugs are found in the second system, the tendency is to turn it off—after all, the original system didn't have those bugs. This dramatically reduces the exposure and overall mileage that the second system can obtain toward being truly production ready. Just like the feature debt that accumulates as the second system is built, the production readiness debt accrues, and it is very hard for the second system to catch up.

Given that there are probably design problems or technical debt in your distributed system that you need or want to fix, and the "second system" approach is often doomed to failure, what can you do? The solution lies in the nature of the distributed system itself. Rather than replacing the entire distributed system, focusing on improving individual microservices can make real progress tractable and significantly reduce the costs of running two systems in production at once. Similarly, it makes sense to invest in abstractions that separate the core business logic from code that may need to change as you grow and scale, for example, the storage layer. This separation means that you can iterate your core systems like storage independently from your business logic and even run two different storage layers side by side for evaluation and testing.

Much like many other aspects of our lives, the urge to throw away the old and build something new, while tempting, is ultimately far less successful than adapting and improving our existing systems to meet the new needs of our businesses and our users.

Summary

This chapter is unlike any of the others; instead of discussing how to build, it discusses how *not* to build. Hopefully, seeing the common ways that distributed systems fail helps you find problems in your existing systems and also to build more reliable new systems. Learning from the failures of others prevents you from making these common mistakes in your systems.

Previous chapter

< 15. AI Inference and Serving

Next chapter

Conclusion: A New Beginning? >